UNIVERSIDAD CATÓLICA DE EL SALVADOR

FACULTAD DE INGENIERÍA Y ARQUITECTURA

ESCUELA DE INGENIERÍA EN DESARROLLO DE SOFTWARE

Gestión de Seguridad en Sistemas Informáticos — Sección "A"

INFORME TÉCNICO DE VULNERABILIDAD: CROSS-SITE SCRIPTING (XSS - CWE-79)

Análisis Teórico Profundo, Mecánica de Parsers, Diagramas de Flujo y Mitigación en Código Seguro

RESPONSABLES DE INVESTIGACIÓN:

- Dennis Ademir Guevara Martínez | Carnet: 2023-GM-607
- Mario Edgardo Villeda Alabi | Carnet: 2023-VA-651
- Luis Felipe Balcaceres Silvestre | Carnet: 2023-BS-602

SANTA ANA, EL SALVADOR — NOVIEMBRE DE 2026

EVALUACIÓN DE PARCIAL 3

1. INTRODUCCIÓN Y JUSTIFICACIÓN TÉCNICA

En el ecosistema del desarrollo web contemporáneo, la seguridad de las aplicaciones del lado del cliente representa uno de los mayores desafíos operacionales para las organizaciones. Entre las vulnerabilidades que aquejan a la capa de presentación, el **Cross-Site Scripting (XSS)**, catalogado por el consorcio MITRE bajo el identificador **CWE-79** (*"Neutralización inadecuada de entradas durante la generación de páginas web"*), ha mantenido una presencia ininterrumpida dentro de las principales categorías de riesgo de OWASP a lo largo de más de dos décadas.

El principio fundamental detrás de una debilidad XSS consiste en la incapacidad del navegador web para distinguir si un fragmento de código JavaScript ejecutable proviene del desarrollador legítimo del sitio o si fue incrustado por un tercero a través de una entrada de datos no confiable. Una vez que el navegador interpreta el script en el contexto de seguridad del origen (*Same-Origin Policy*), el código malicioso adquiere acceso irrestricto a los recursos locales del usuario, permitiendo el secuestro de credenciales de sesión (*Session Hijacking*), la alteración visual del sitio (*Defacement*), el redireccionamiento a portales fraudulentos y la ejecución de operaciones no autorizadas en nombre de la víctima.

2. ANATOMÍA Y MECÁNICA PROFUNDA DE CWE-79

Para comprender por qué XSS es tan evasivo y prevalente, es necesario examinar el funcionamiento interno del **Analizador HTML (HTML Parser)** del navegador web. El navegador opera bajo una máquina de estados determinista (según la especificación de la W3C/WHATWG). Inicialmente, el analizador procesa texto en el *Data State*. Sin embargo, al encontrar el delimitador menor que (<), transiciona inmediatamente al *Tag Open State*. Si el contenido de la etiqueta corresponde a `<script>`, el motor suspende la construcción del árbol DOM y transfiere el flujo de control al motor de ejecución de JavaScript (como V8 en Chrome o SpiderMonkey en Firefox).

La falla se consuma cuando una aplicación web mezcla indiscriminadamente **datos no confiables** con **instrucciones de control**. Al no codificar adecuadamente los delimitadores sintácticos, los datos del usuario son "promovidos" a instrucciones de ejecución por el analizador léxico.

2.1 Los Cuatro Contextos Sintácticos de Inyección

El riesgo y la técnica de explotación varían sustancialmente según el contexto sintáctico donde se renderiza la entrada:

Contexto de Inyección	Ejemplo de Código Vulnerable	Vector de Ataque Típico	Mecanismo de Ruptura
1. Cuerpo HTML	`<p>Hola <?php echo $_GET['user']; ?> </p>`	`<script>alert(1)</script>`	Inyecta una nueva etiqueta de primer nivel en el árbol DOM.

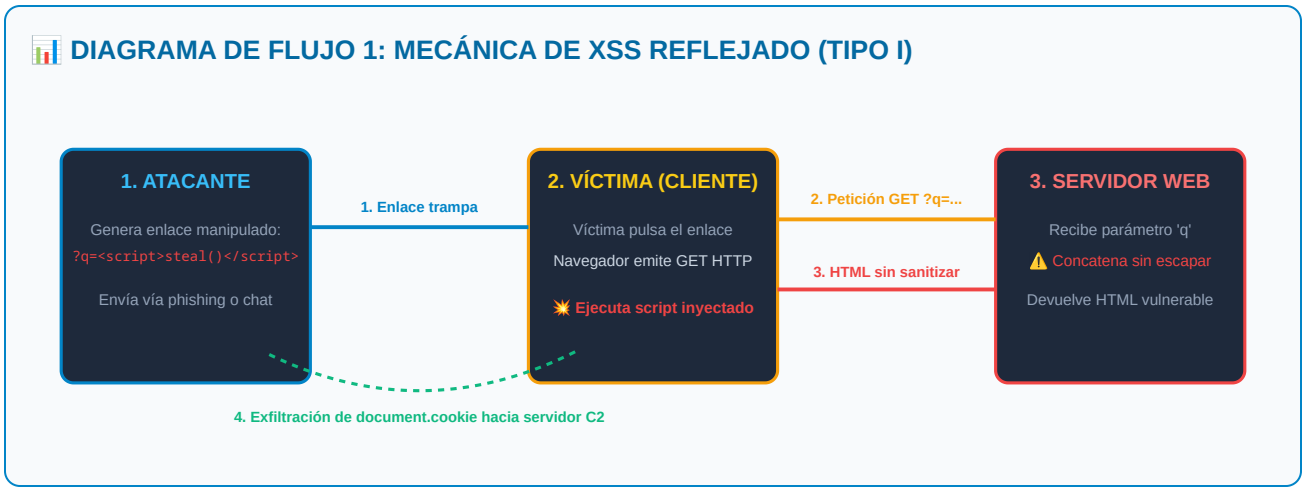
(HTML Body)			
2. Atributo HTML	`<input value="<?php echo $_GET['user']; ?>">`	`" onfocus="alert(1)" autofocus="`	Cierra las comillas del atributo e inyecta un manejador de eventos en línea.
3. Bloque JavaScript	`<script>var id = '<?php echo $_GET['id']; ?>'; </script>`	`'; alert(1); // o </script>`	Rompe el literal de cadena de JS o cierra prematuramente el bloque de script.
4. Esquema de URL	`<a href="<?php echo $_GET['url']; ?>">Ir</a>`	`javascript:alert(document.cookie)`	Inyecta un pseudoprotocolo ejecutable en lugar de una URL web válida.

El dilema del Same-Origin Policy (SOP): La Política de Mismo Origen es la principal barrera de contención del navegador: prohíbe que scripts de `atacante.com` lean datos de `banco.com`. Sin embargo, XSS representa la neutralización total del SOP, ya que el script malicioso se ejecuta **físicamente dentro del dominio de la víctima**, obteniendo acceso total a variables globales, APIs asíncronas (`fetch`, `XMLHttpRequest`), WebStorage (`localStorage`, `sessionStorage`) y `document.cookie`.

3. TAXONOMÍA Y DIAGRAMAS DE FLUJO POR TIPO

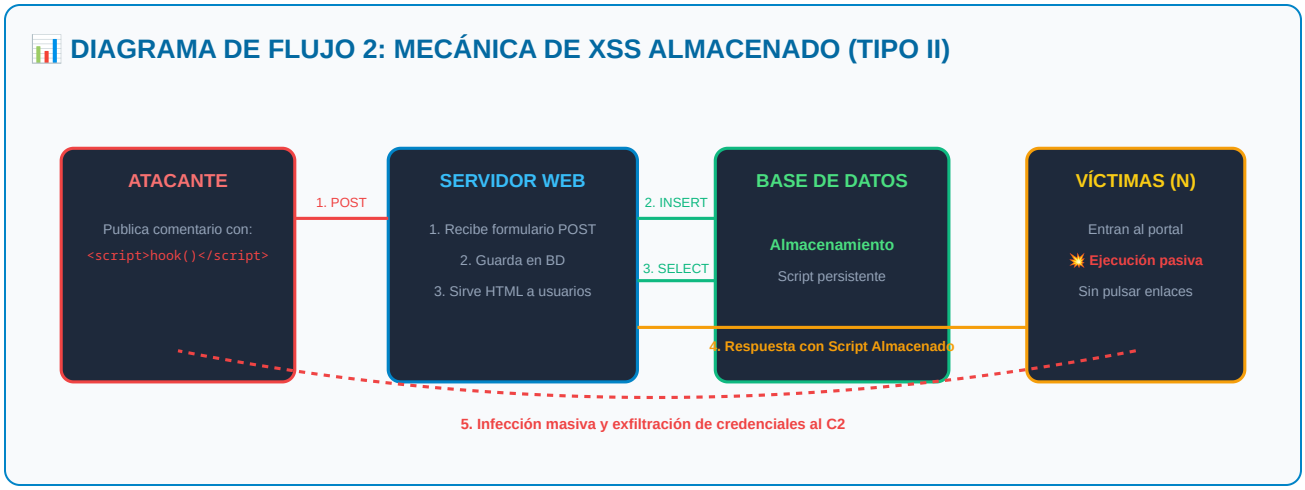
3.1 XSS Reflejado (Tipo I - No Persistente)

La carga maliciosa viaja dentro de la propia petición HTTP (típicamente en parámetros GET de cadenas de consulta o en formularios POST inmediatos) y el servidor web la incluye de regreso en el documento HTML sin escapar. El script no reside en la base de datos; su ciclo de vida dura exactamente lo que tarda la petición en procesarse. Depende necesariamente de técnicas de ingeniería social para persuadir a la víctima de que abra un hipervínculo manipulado.



3.2 XSS Almacenado o Persistente (Tipo II - Máxima Severidad)

Representa la variante más devastadora. La carga maliciosa es recibida por el servidor y almacenada permanentemente en los repositorios de persistencia (bases de datos relacionales, NoSQL, logs del sistema o foros). Cada vez que cualquier usuario autenticado navega hacia la sección comprometida, el servidor extrae la inyección y la sirve embebida en la página web legítima, provocando una ejecución masiva y desatendida.

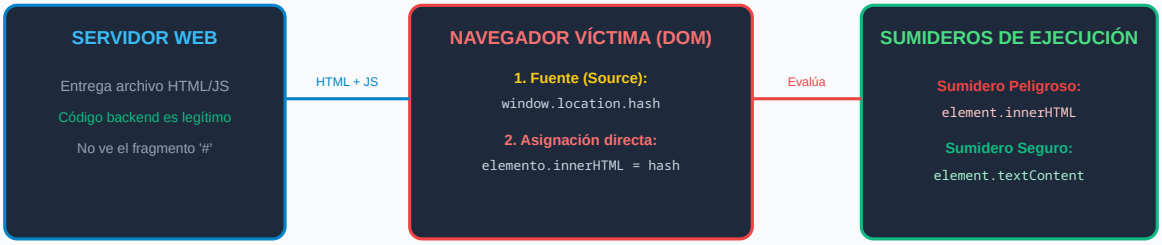


3.3 XSS Basado en DOM (Tipo 0 / Client-Side XSS)

A diferencia de los tipos anteriores, en el XSS basado en DOM el servidor web **no participa en la vulnerabilidad ni recibe la carga maliciosa en sus registros HTTP**. Esto se debe a que la carga viaja en el identificador de fragmento de la URL (el caracter almohadilla `#`), el cual los navegadores nunca envían al servidor en la cabecera de petición.

El fallo reside enteramente en el código JavaScript del frontend: un script cliente lee datos de una **Fuente** (*Source*) insegura (como `location.hash`, `location.search` o `document.referrer`) y los asigna sin validación a un **Sumidero Peligroso** (*Execution Sink*) como `innerHTML`, `document.write()` o `eval()`.

📊 DIAGRAMA DE FLUJO 3: MECÁNICA DE XSS BASADO EN DOM (TIPO 0)



4. IMPACTO EN LA TRIADA DE SEGURIDAD (CIA)

Pilar	Nivel	Manifestación en Ataques XSS
Confidencialidad	**ALTO**	Exfiltración de credenciales de sesión, tokens JWT, información personal ingresada en formularios y lectura de datos privados en el DOM.
Integridad	**CRÍTICO**	Desfiguración del portal (*Defacement*), alteración de transacciones en curso, inserción de falsos campos de autenticación (*Phishing in-site*).
Disponibilidad	**MEDIO**	Generación de bucles infinitos en el hilo principal de JavaScript, bloqueo del navegador del usuario final y denegación de servicio local.

5. ESTRATEGIA INTEGRAL DE DEFENSA EN PROFUNDIDAD

Para neutralizar XSS en sus tres variantes se implementan tres capas defensivas simultáneas:

1. **Codificación Contextual de Salida (Output Encoding):** Utilizar funciones de codificación específicas para cada contexto (`html.escape()` en backend, o bibliotecas como OWASP Java Encoder). Convierte caracteres de control sintáctico (`< > & " '`) en entidades HTML seguras, neutralizando los tipos I y II.
2. **Blindaje de Cookies con HttpOnly (RFC 6265):** Marcar las cookies de sesión con `HttpOnly=True`. El navegador impide que cualquier script acceda a `document.cookie`, neutralizando el secuestro de sesiones (*Session Hijacking*).
3. **Content Security Policy (CSP Nivel 3):** Emitir la cabecera `Content-Security-Policy: default-src 'self'`, lo que inhabilita scripts en línea (*unsafe-inline*) y `eval()`, deteniendo la ejecución de inyecciones reflejadas, almacenadas o de DOM.

6. CONCLUSIONES Y REFERENCIAS

• El análisis demostró que XSS no es un problema de red, sino de falta de distinción entre datos e instrucciones en el cliente.
• La combinación de codificación contextual de salida, blindaje de cookies con `HttpOnly` y directivas CSP ofrece una protección resiliente y comprobada contra las tres variantes de CWE-79.
• *Referencias: OWASP Top 10 (A03:2021 Injection), MITRE CWE-79, NIST SP 800-53 Rev. 5, RFC 6265.*